

Язык программирования C++

Лекция 10: конструктор копирования, оператор присваивания, const, перегрузка операторов

Никита Лисица

2026

Вспоминаем прошлую лекцию

```
class Array {  
private:  
    size_t size;  
    int * data;  
  
public:  
    Array();  
    Array(size_t size);  
    ~Array();  
  
    int get(size_t index);  
    void set(size_t index, int value);  
};
```

Конструктор копирования (copy constructor)

- Часто полезно создать копию объекта

Конструктор копирования (copy constructor)

- Часто полезно создать копию объекта
- Можно сделать для этого отдельный метод:

```
class Array {  
    ...  
public:  
    ...  
    Array copy();  
};
```

Конструктор копирования (copy constructor)

- Часто полезно создать копию объекта
- Можно сделать для этого отдельный метод:

```
class Array {  
    ...  
public:  
    ...  
    Array copy();  
};
```

- Но иногда хочется пользоваться объектами класса как обычными переменными, и использовать обычный синтаксис копирования:

```
Array a(10);  
Array b = a;
```

Конструктор копирования (copy constructor)

- По умолчанию объекты классов тоже умеют копироваться, как структуры, т.е. просто копируя все поля

Конструктор копирования (copy constructor)

- По умолчанию объекты классов тоже умеют копироваться, как структуры, т.е. просто копируя все поля

```
Array a(10);  
Array b = a;
```

- Где проблема?

Конструктор копирования (copy constructor)

- По умолчанию объекты классов тоже умеют копироваться, как структуры, т.е. просто копируя все поля

```
Array a(10);  
Array b = a;
```

- Где проблема? Оба объекта ссылаются на один и тот же массив с данными

Конструктор копирования (copy constructor)

- По умолчанию объекты классов тоже умеют копироваться, как структуры, т.е. просто копируя все поля

```
Array a(10);  
Array b = a;
```

- Где проблема? Оба объекта ссылаются на один и тот же массив с данными
- $\implies$ Изменение данных в одном поменяет и в другом

Конструктор копирования (copy constructor)

- По умолчанию объекты классов тоже умеют копироваться, как структуры, т.е. просто копируя все поля

```
Array a(10);  
Array b = a;
```

- Где проблема? Оба объекта ссылаются на один и тот же массив с данными
- $\implies$ Изменение данных в одном поменяет и в другом
- $\implies$ Оба в деструкторе попробуют его удалить

Конструктор копирования (copy constructor)

- Решение: конструктор копирования

```
class Array {  
    ...  
public:  
    ...  
    // Конструктор копирования  
    Array(const Array& other) {  
        size = other.size;  
        data = new int [size];  
        for (size_t i = 0; i < size; ++i)  
            data[i] = other.data[i];  
    }  
};
```

Конструктор копирования (copy constructor)

- Решение: конструктор копирования

```
class Array {  
    ...  
public:  
    ...  
    // Конструктор копирования  
    Array(const Array& other) {  
        size = other.size;  
        data = new int [size];  
        for (size_t i = 0; i < size; ++i)  
            data[i] = other.data[i];  
    }  
};
```

```
// Вызывается конструктор от size_t  
Array a(10);  
  
// Вызывается конструктор копирования  
Array b = a;
```

Конструктор копирования (copy constructor)

- **NB:** Передача в функцию по значению или `return` – тоже вызов копирования:

```
Array makeArray() {  
    ...  
    // Вызов конструктора копирования для возвращаемого значения  
    return a;  
}
```

Конструктор копирования (copy constructor)

- **NB:** Передача в функцию по значению или `return` – тоже вызов копирования:

```
Array makeArray() {  
    ...  
    // Вызов конструктора копирования для возвращаемого значения  
    return a;  
}
```

```
void print(Array a) {  
    ...  
}  
  
Array a(10);  
  
// Вызов конструктора копирования для аргумента функции  
print(a);
```

Конструктор копирования (copy constructor)

- Что не так в этом коде?

```
void print(Array a) {  
    for (size_t i = 0; i < a.get_size(); i++) {  
        printf("%d,", a.get(i));  
    }  
}
```

Конструктор копирования (copy constructor)

- Что не так в этом коде?

```
void print(Array a) {  
    for (size_t i = 0; i < a.get_size(); i++) {  
        printf("%d,", a.get(i));  
    }  
}
```

- Функция использует массив только для чтения $\implies$ незачем делать копию (это не бесплатно), лучше передать по константной ссылке:

```
void print(const Array& a) {  
    ...  
}
```


Конструктор копирования (copy constructor)

- Почему конструктор копирования не объявить вот так?

```
class Array {  
    ...  
public:  
    Array(Array other);  
}
```

Конструктор копирования (copy constructor)

- Почему конструктор копирования не объявить вот так?

```
class Array {  
    ...  
public:  
    Array(Array other);  
}
```

- Массив **other** тоже надо как-то передать внутрь конструктора копирования

Конструктор копирования (copy constructor)

- Почему конструктор копирования не объявить вот так?

```
class Array {  
    ...  
public:  
    Array(Array other);  
}
```

- Массив **other** тоже надо как-то передать внутрь конструктора копирования
- $\implies$ Бесконечная рекурсия :)

Оператор присваивания (copy assignment)

- В чём разница между этими двумя строками?

```
/* 1 */ int b = a;  
/* 2 */ b = a;
```

Оператор присваивания (copy assignment)

- В чём разница между этими двумя строками?

```
/* 1 */ int b = a;  
/* 2 */ b = a;
```

- В первом случае объект *создаётся*, и *инициализируется* копией **b**

Оператор присваивания (copy assignment)

- В чём разница между этими двумя строками?

```
/* 1 */ int b = a;  
/* 2 */ b = a;
```

- В первом случае объект *создаётся*, и *инициализируется* копией **b**
- Во втором случае объект *уже создан*, и *перезаписывается* копией **b**

Оператор присваивания (copy assignment)

- В чём разница между этими двумя строками?

```
/* 1 */ int b = a;  
/* 2 */ b = a;
```

- В первом случае объект *создаётся*, и *инициализируется* копией `b`
- Во втором случае объект *уже создан*, и *перезаписывается* копией `b`
- Не очень важно для примитивных типов, но важно для классов!

Оператор присваивания (copy assignment)

```
Array a(10), b(20);  
  
// Перезапись копированием  
b = a;
```


Оператор присваивания (copy assignment)

```
Array a(10), b(20);  
  
// Перезапись копированием  
b = a;
```

- Так же как с копированием, по умолчанию просто скопируются все поля $\Rightarrow$ все те же проблемы, + утечка памяти массива **b**

Оператор присваивания (copy assignment)

```
// array.h
class Array {
    ...
public:
    // Оператор присваивания
    void operator = (const Array& other);
};
```

```
// array.cpp

// Для разнообразия, реализовали в .cpp
void Array::operator = (const Array& other) {
    delete [] data;
    size = other.size;
    data = new int [size];
    for (size_t i = 0; i < size; ++i)
        data[i] = other.data[i];
}
```

Оператор присваивания (copy assignment)

- Более идиоматичный вариант:

```
Array& Array::operator = (const Array& other) {  
    if (&other == this) {  
        return *this;  
    }  
    delete [] data;  
    size = other.size;  
    data = new int [size];  
    for (size_t i = 0; i < size; ++i)  
        data[i] = other.data[i];  
    return *this;  
}
```

Оператор присваивания (copy assignment)

- Более идиоматичный вариант:

```
Array& Array::operator = (const Array& other) {  
    if (&other == this) {  
        return *this;  
    }  
    delete [] data;  
    size = other.size;  
    data = new int [size];  
    for (size_t i = 0; i < size; ++i)  
        data[i] = other.data[i];  
    return *this;  
}
```

- Зачем проверка `&other == this`?

Оператор присваивания (copy assignment)

- Более идиоматичный вариант:

```
Array& Array::operator = (const Array& other) {  
    if (&other == this) {  
        return *this;  
    }  
    delete [] data;  
    size = other.size;  
    data = new int [size];  
    for (size_t i = 0; i < size; ++i)  
        data[i] = other.data[i];  
    return *this;  
}
```

- Зачем проверка `&other == this`? Для самоприсваивания:
`a = a;`

Оператор присваивания (copy assignment)

- Более идиоматичный вариант:

```
Array& Array::operator = (const Array& other) {  
    if (&other == this) {  
        return *this;  
    }  
    delete [] data;  
    size = other.size;  
    data = new int [size];  
    for (size_t i = 0; i < size; ++i)  
        data[i] = other.data[i];  
    return *this;  
}
```

- Зачем проверка `&other == this`? Для самоприсваивания:
`a = a;`
- Зачем возвращать `*this`?

Оператор присваивания (copy assignment)

- Более идиоматичный вариант:

```
Array& Array::operator = (const Array& other) {  
    if (&other == this) {  
        return *this;  
    }  
    delete [] data;  
    size = other.size;  
    data = new int [size];  
    for (size_t i = 0; i < size; ++i)  
        data[i] = other.data[i];  
    return *this;  
}
```

- Зачем проверка `&other == this`? Для самоприсваивания:
`a = a;`
- Зачем возвращать `*this`? Например, для *chained assignment*:
`a = b = c;`

Явные/неявные конструкторы

- С конструктором `Array(size_t)` есть проблема:

```
void print(Array array) { ... }

// Эквивалентно print(Array(10))
// Происходит неявное преобразование
// size_t -> Array
print(10);

// Эквивалентно Array a(10)
Array a = 10;

Array someFunc() {
    // Тоже неявное преобразование
    return 10;
}
```


Явные/неявные конструкторы

- С конструктором `Array(size_t)` есть проблема:

```
void print(Array array) { ... }

// Эквивалентно print(Array(10))
// Происходит неявное преобразование
// size_t -> Array
print(10);

// Эквивалентно Array a(10)
Array a = 10;

Array someFunc() {
    // Тоже неявное преобразование
    return 10;
}
```

- Инициализация присваиванием `a = 10` выглядит странно

Явные/неявные конструкторы

- С конструктором `Array(size_t)` есть проблема:

```
void print(Array array) { ... }

// Эквивалентно print(Array(10))
// Происходит неявное преобразование
// size_t -> Array
print(10);

// Эквивалентно Array a(10)
Array a = 10;

Array someFunc() {
    // Тоже неявное преобразование
    return 10;
}
```

- Инициализация присваиванием `a = 10` выглядит странно
- Вызов `print(10)` может делать вообще не то, что мы задумали, особенно с учётом перегрузки функций

Явные/неявные конструкторы

- С конструктором `Array(size_t)` есть проблема:

```
void print(Array array) { ... }

// Эквивалентно print(Array(10))
// Происходит неявное преобразование
// size_t -> Array
print(10);

// Эквивалентно Array a(10)
Array a = 10;

Array someFunc() {
    // Тоже неявное преобразование
    return 10;
}
```

- Инициализация присваиванием `a = 10` выглядит странно
- Вызов `print(10)` может делать вообще не то, что мы задумали, особенно с учётом перегрузки функций
- `return 10;` – скорее всего, опечатка

Явные/неявные конструкторы

- Решение: конструктор можно пометить как **явный (explicit)**:

```
class Array {  
public:  
    explicit Array(size_t size);  
    ...  
};  
  
// ОК, это явный вызов конструктора  
Array a(10);  
  
// Ошибка компиляции  
Array a = 10;  
  
// Ошибка компиляции: неявное  
// преобразование запрещено  
print(10);  
  
Array someFunc() {  
    // Ошибка компиляции  
    return 10;  
}
```

(N)RVO, copy elision

- Во что компилируется такой код?

```
Array makeArray() {  
    ...  
    return Array(10);  
}
```

(N)RVO, copy elision

- Во что компилируется такой код?

```
Array makeArray() {  
    ...  
    return Array(10);  
}
```

- Функция, вызывающая `makeArray()`, заводит на стеке место под возвращаемое значение, и передаёт его адрес в `makeArray()`
- `makeArray()` создаёт локальный объект `Array(10)`, затем *копирует* его в возвращаемое значение

(N)RVO, copy elision

- Во что компилируется такой код?

```
Array makeArray() {  
    ...  
    return Array(10);  
}
```

- Функция, вызывающая `makeArray()`, заводит на стеке место под возвращаемое значение, и передаёт его адрес в `makeArray()`
- `makeArray()` создаёт локальный объект `Array(10)`, затем *копирует* его в возвращаемое значение
- Временный объект `Array(10)` после этого сразу удалится $\implies$ можно его создать сразу на месте возвращаемого значения

(N)RVO, copy elision

- Во что компилируется такой код?

```
Array makeArray() {  
    ...  
    return Array(10);  
}
```

- Функция, вызывающая `makeArray()`, заводит на стеке место под возвращаемое значение, и передаёт его адрес в `makeArray()`
- `makeArray()` создаёт локальный объект `Array(10)`, затем копирует его в возвращаемое значение
- Временный объект `Array(10)` после этого сразу удалится $\implies$ можно его создать сразу на месте возвращаемого значения
- Эта оптимизация называется (U)RVO – *(Unnamed) Return Value Optimization*

- Вариант чуть сложнее:

```
Array makeArray() {  
    Array array(10);  
    ...  
    return array;  
}
```

- Вариант чуть сложнее:

```
Array makeArray() {  
    Array array(10);  
    ...  
    return array;  
}
```

- Здесь объект создаётся задолго до **return**, тоже копируется в возвращаемое значение и потом не нужен $\implies$ его тоже можно создать сразу на месте возвращаемого значения

(N)RVO, copy elision

- Вариант чуть сложнее:

```
Array makeArray() {  
    Array array(10);  
    ...  
    return array;  
}
```

- Здесь объект создаётся задолго до **return**, тоже копируется в возвращаемое значение и потом не нужен $\implies$ его тоже можно создать сразу на месте возвращаемого значения
- Эта оптимизация называется NRVO – *Named Return Value Optimization*

- Общее название для этих оптимизаций – *copy elision*

- Общее название для этих оптимизаций – *copy elision*
- Их делают компиляторы за нас :)

- Общее название для этих оптимизаций – *copy elision*
- Их делают компиляторы за нас :)
- Начиная с C++17 URVO обязательна (т.е. всегда вставляется компилятором)

- Общее название для этих оптимизаций – *copy elision*
- Их делают компиляторы за нас :)
- Начиная с C++17 URVO обязательна (т.е. всегда вставляется компилятором)
- После C++11 большинство случаев *copy elision* свелись к *implicit move*, об этом позже

- Общее название для этих оптимизаций – *copy elision*
- Их делают компиляторы за нас :)
- Начиная с C++17 URVO обязательна (т.е. всегда вставляется компилятором)
- После C++11 большинство случаев *copy elision* свелись к *implicit move*, об этом позже
- Про это важно помнить, так как компилятор может убрать вызов копирования даже если в нём есть побочные эффекты!

- Если у класса есть поля, любой его конструктор должен проинициализировать все поля (т.е. вызвать их конструкторы)

Конструкторы полей

- Если у класса есть поля, любой его конструктор должен проинициализировать все поля (т.е. вызвать их конструкторы)
- По умолчанию, вызовется дефолтный конструктор (который для примитивных типов ничего не делает):

```
Array::Array(size_t size) {  
    // Технически, к этому моменту вызвались дефолтные  
    // конструкторы size и data!  
    this->size = size;  
    data = new int [size];  
}
```

Конструкторы полей

- Если у класса есть поля, любой его конструктор должен проинициализировать все поля (т.е. вызвать их конструкторы)
- По умолчанию, вызовется дефолтный конструктор (который для примитивных типов ничего не делает):

```
Array::Array(size_t size) {  
    // Технически, к этому моменту вызвались дефолтные  
    // конструкторы size и data!  
    this->size = size;  
    data = new int [size];  
}
```

- Что, если у поля нет дефолтного конструктора?

Конструкторы полей

```
class Vector3 {
public:
    Vector3(float x, float y, float z);
};

class RigidBody {
private:
    Vector3 position;
public:
    RigidBody() {
        // Ошибка: у поля position нет дефолтного конструктора
    }
};
```

Конструкторы полей

- Есть специальный синтаксис для инициализации полей:

```
class RigidBody {  
private:  
    Vector3 position;  
public:  
    RigidBody() : position(0.f, 0.f, 0.f) {  
        // Не ошибка: у поля position явно вызван конструктор  
    }  
};
```

Конструкторы полей

- Если полей несколько, перечисляем через запятую (в порядке их объявления):

```
class RigidBody {  
private:  
    Vector3 position;  
    Quaternion rotation;  
public:  
    RigidBody(float zAngle)  
        : position(0.f, 0.f, 0.f)  
        , rotation(cos(zAngle / 2.f), 0.f, 0.f, sin(zAngle / 2.f))  
    {}  
};
```

Конструкторы полей

- Если полей несколько, перечисляем через запятую (в порядке их объявления):

```
class RigidBody {  
private:  
    Vector3 position;  
    Quaternion rotation;  
public:  
    RigidBody(float zAngle)  
        : position(0.f, 0.f, 0.f)  
        , rotation(cos(zAngle / 2.f), 0.f, 0.f, sin(zAngle / 2.f))  
    {}  
};
```

- Если какое-то поле пропущено, у него вызывается дефолтный конструктор

Конструкторы полей

- Если полей несколько, перечисляем через запятую (в порядке их объявления):

```
class RigidBody {  
private:  
    Vector3 position;  
    Quaternion rotation;  
public:  
    RigidBody(float zAngle)  
        : position(0.f, 0.f, 0.f)  
        , rotation(cos(zAngle / 2.f), 0.f, 0.f, sin(zAngle / 2.f))  
    {}  
};
```

- Если какое-то поле пропущено, у него вызывается дефолтный конструктор
- В теле (коде) конструктора все поля класса **уже созданы и проинициализированы!**

Напоминание: const в языке C

- Нужен, чтобы более чётко описывать интерфейсы и обезопасить себя от ошибок:

```
const int pi = 3;
// Ошибка компиляции
pi = 4;

int i = 10;
const int * ptr = &i;
// Ошибка компиляции
*ptr = 42;

// Ошибка компиляции
int * ptr2 = &pi;

// Явно говорит, что функция не меняет
// данные в переданном массиве
size_t strlen(const char* str);
```

Напоминание: const в языке C

```
// Это инициализация массива набором символов
char str1[] = "Hello, world!";

// Это инициализация указателя адресом в статической памяти
const char* str2 = "Hello, world!";

// Ошибка компиляции
char* str3 = "Hello, world!";

// Вероятнее всего, runtime креш
((char*)str2)[0] = "X";
```

Классы и const: константное поле

- Поля класса могут быть константными, их нельзя менять:

```
class Array {  
private:  
    const size_t size;  
    ...  
public:  
    void resize(size_t new_size) {  
        // Ошибка компиляции: size - константное поле  
        size = new_size;  
    }  
};
```

Классы и const: константное поле

- Константное поле обязательно нужно явно проинициализировать в конструкторе:

```
class Array {  
private:  
    const size_t size;  
    ...  
public:  
    Array() {  
        // Ошибка компиляции: size не проинициализирован  
    }  
  
    Array(size_t size)  
        // ОК: первый size это поле, второй - аргумент конструктора  
        : size(size)  
    {}  
};
```

Классы и const: константный метод

- Методы тоже могут быть константными – их можно вызывать у константных объектов класса

Классы и const: константный метод

- Методы тоже могут быть константными – их можно вызывать у константных объектов класса
- По смыслу это методы, не меняющие состояние класса

Классы и const: КОНСТАНТНЫЙ МЕТОД

- Методы тоже могут быть константными – их можно вызывать у константных объектов класса
- По смыслу это методы, не меняющие состояние класса

```
class Array {  
private:  
    size_t size;  
    ...  
public:  
    size_t get_size() const {  
        return size;  
    }  
  
    int get(size_t index) const {  
        return data[index];  
    }  
  
    // He const  
    void set(size_t index, int value) {  
        data[index] = value;  
    }  
};
```

Классы и const: константный метод

```
const Array array(10);  
  
// OK, константный метод  
array.get_size();  
  
// OK, константный метод  
array.get(5);  
  
// Ошибка компиляции  
array.set(5, 67);
```


Классы и const: константный метод

```
const Array array(10);  
  
// OK, константный метод  
array.get_size();  
  
// OK, константный метод  
array.get(5);  
  
// Ошибка компиляции  
array.set(5, 67);
```

- Константные методы можно вызывать у любых объектов

Классы и const: константный метод

```
const Array array(10);  
  
// OK, константный метод  
array.get_size();  
  
// OK, константный метод  
array.get(5);  
  
// Ошибка компиляции  
array.set(5, 67);
```

- Константные методы можно вызывать у любых объектов
- Не-константные методы можно вызывать только у не-константных объектов

Классы и const: mutable

- Рассмотрим пример кода – данные, которые лениво (при первом обращении) читаются с диска:

```
class InputData {  
private:  
    Array values;  
public:  
    const Array& get() const {  
        if (values.empty()) {  
            InputFile file(...);  
            values.resize(file.readNext());  
            for (size_t i = 0; i < values.size(); ++i)  
                values.set(i, file.readNext());  
        }  
  
        return values;  
    }  
}
```

Классы и const: mutable

- Рассмотрим пример кода – данные, которые лениво (при первом обращении) читаются с диска:

```
class InputData {  
private:  
    Array values;  
public:  
    const Array& get() const {  
        if (values.empty()) {  
            InputFile file(...);  
            values.resize(file.readNext());  
            for (size_t i = 0; i < values.size(); ++i)  
                values.set(i, file.readNext());  
        }  
  
        return values;  
    }  
}
```

- Ошибка компиляции: нельзя модифицировать поле `values` в константном методе

- Решение №1: убрать `const` у метода `get()`

- Решение №1: убрать `const` у метода `get()` – не позволит передавать `InputData` по константной ссылке и т.п.

- Решение №1: убрать `const` у метода `get()` – не позволит передавать `InputData` по константной ссылке и т.п.
- Решение №2: ключевое слово `mutable`

- `mutable` у поля класса означает, что это поле можно менять даже из константных методов класса

Классы и const: mutable

- **mutable** у поля класса означает, что это поле можно менять даже из константных методов класса

```
class InputData {  
private:  
    mutable Array values;  
public:  
    const Array& get() const {  
        if (values.empty()) {  
            InputFile file(...);  
            // OK: mutable поле можно менять  
            values.resize(file.readNext());  
            ...  
        }  
  
        return values;  
    }  
}
```

- Почти всегда считается, что использование `mutable` – плохой тон, и усложняет чтение кода

- Почти всегда считается, что использование `mutable` – плохой тон, и усложняет чтение кода
- Два основных применения, где без `mutable` часто не обойтись: *кеширование* и *мьютексы*

- Почти всегда считается, что использование `mutable` – плохой тон, и усложняет чтение кода
- Два основных применения, где без `mutable` часто не обойтись: *кеширование* и *мьютексы*
- **NB:** Ещё бывают `mutable` лямбды, это не плохой тон, но о них позже :)

Операторы

- В C++ очень много операторов (≥ 60), вот некоторые из них:

```
a++ ++a a-- --a +a -a !a ~a
a() a[] a.b a->b a.*b *a &a
a+b a-b a*b a/b a%b a>>b a<<b
a&b a|b a^b a&&b a||b
a==b a!=b a<b a<=b a>b a>=b a<=>b
a=b a+=b a-=b a*=b a/=b a%=b
a&b a|b a^b a>>=b a<<=b
a?b:c a,b
```

Операторы

- В C++ очень много операторов (≥ 60), вот некоторые из них:

```
a++ ++a a-- --a +a -a !a ~a
a() a[] a.b a->b a.*b *a &a
a+b a-b a*b a/b a%b a>>b a<<b
a&b a|b a^b a&&b a||b
a==b a!=b a<b a<=b a>b a>=b a<=>b
a=b a+=b a-=b a*=b a/=b a%=b
a&b a|=b a^=b a>>=b a<<=b
a?b:c a,b
```

- Как и функции, их можно перегружать, но только если один из аргументов – пользовательский тип (т.е. класс)

Операторы

- В C++ очень много операторов (≥ 60), вот некоторые из них:

```
a++ ++a a-- --a +a -a !a ~a
a() a[] a.b a->b a.*b *a &a
a+b a-b a*b a/b a%b a>>b a<<b
a&b a|b a^b a&&b a||b
a==b a!=b a<b a<=b a>b a>=b a<=>b
a=b a+=b a-=b a*=b a/=b a%=b
a&b a|=b a^=b a>>=b a<<=b
a?b:c a,b
```

- Как и функции, их можно перегружать, но только если один из аргументов – пользовательский тип (т.е. класс)
- Перегружать можно почти всё, исключения – оператор доступа **a.b**, оператор доступа по указателю на поле **a.*b**, тернарный условный оператор **a?b:c**

- Рассмотрим класс длинной арифметики **BigInt**: как для него определить оператор сложения? Два варианта:

Перегрузка операторов

- Рассмотрим класс длинной арифметики **BigInt**: как для него определить оператор сложения? Два варианта:
- Как свободную функцию

```
BigInt operator+(const BigInt& a, const BigInt& b) {  
    ...  
}
```

Перегрузка операторов

- Рассмотрим класс длинной арифметики `BigInt`: как для него определить оператор сложения? Два варианта:
- Как свободную функцию

```
BigInt operator+(const BigInt& a, const BigInt& b) {  
    ...  
}
```

- Как особый метод класса

```
BigInt BigInt::operator+(const BigInt& b) {  
    // Здесь левый аргумент это this  
    ...  
}
```

```
BigInt a, b;  
...  
// Вызывает operator+(a, b) либо a.operator+(b)  
// В зависимости от того, кто из них определён  
BigInt c = a + b;
```

Перегрузка операторов

- Как лучше: внутри класса `BigInt::operator+(BigInt)` или снаружи `operator+(BigInt, BigInt)`?

Перегрузка операторов

- Как лучше: внутри класса `BigInt::operator+(BigInt)` или снаружи `operator+(BigInt, BigInt)`?
- Пусть есть конструктор `BigInt(int)`. Рассмотрим ситуации:

```
BigInt a = 10, b = 20;

// ОК в любом случае
a + b;

// ОК в любом случае
a + 3;

// Не ОК в случае метода: operator+ объявлен методом в классе
// BigInt и не имеет отношения к типу int
3 + a;

// Метода int::operator+(BigInt) быть не может,
// так как int - не класс :)
```

- В целом, часто полезно придерживаться следующего правила:

- В целом, часто полезно придерживаться следующего правила:
 - Если функции нужно знание о внутренностях класса, то это должен быть метод

- В целом, часто полезно придерживаться следующего правила:
 - Если функции нужно знание о внутренностях класса, то это должен быть метод
 - Если функции достаточно публичного интерфейса класса, то лучше её сделать свободной функцией, принимающей объект класса (например, по ссылке)

- В целом, часто полезно придерживаться следующего правила:
 - Если функции нужно знание о внутренностях класса, то это должен быть метод
 - Если функции достаточно публичного интерфейса класса, то лучше её сделать свободной функцией, принимающей объект класса (например, по ссылке)
- Это облегчает чтение кода класса и проверку его инвариантов

Метод vs функция

- В целом, часто полезно придерживаться следующего правила:
 - Если функции нужно знание о внутренностях класса, то это должен быть метод
 - Если функции достаточно публичного интерфейса класса, то лучше её сделать свободной функцией, принимающей объект класса (например, по ссылке)
- Это облегчает чтение кода класса и проверку его инвариантов
- **NB:** Применимо не всегда, иногда удобство использования важнее чистоты кода :)

Compound assignment

- Операторы вида `a += b` называются *compound assignment*

Compound assignment

- Операторы вида $a \ += \ b$ называются *compound assignment*
- $a \ += \ b$ можно реализовать через $a \ + \ b$, а можно и наоборот

Compound assignment

- Операторы вида `a += b` называются *compound assignment*
- `a += b` можно реализовать через `a + b`, а можно и наоборот
- Наоборот (`a + b` через `a += b`) часто оказывается эффективнее, и тогда `a += b` приходится делать методом класса, так как ему нужен доступ к внутренним данным класса:

Compound assignment

- Операторы вида `a += b` называются *compound assignment*
- `a += b` можно реализовать через `a + b`, а можно и наоборот
- Наоборот (`a + b` через `a += b`) часто оказывается эффективнее, и тогда `a += b` приходится делать методом класса, так как ему нужен доступ к внутренним данным класса:

```
BigInt& BigInt::operator+=(const BigInt& other) {  
    // выделяем память, прибавляем, ...  
    return *this;  
}  
  
BigInt operator+(BigInt a, const BigInt& b) {  
    return a += b;  
}
```

Оператор доступа по индексу

```
class Array {  
    ...  
public:  
  
    // Можно было вернуть const int&  
    int operator[](size_t index) const {  
        return data[index];  
    }  
  
    int& operator[](size_t index) {  
        return data[index];  
    }  
}
```

Оператор доступа по индексу

```
class Array {  
    ...  
public:  
  
    // Можно было вернуть const int&  
    int operator[](size_t index) const {  
        return data[index];  
    }  
  
    int& operator[](size_t index) {  
        return data[index];  
    }  
}
```

- Классический паттерн в C++: приходится реализовывать два оператора – const и не-const

Оператор доступа по индексу

- Начиная с C++23 есть `operator[]` с произвольным числом аргументов:

```
class Matrix {  
    ...  
public:  
    float& operator[](size_t row, size_t column);  
};  
  
Matrix m;  
m[3, 4] = 2.718f;
```

Префиксный/постфиксный инкремент/декремент

- Напоминание:

```
int a = 10;  
  
// Префиксный: сначала прибавить, потом вернуть значение  
int b = ++a;  
// Здесь a = 11, b = 11  
  
// Постфиксный: сначала вернуть значение, потом прибавить  
int c = a++;  
// Здесь a = 12, c = 11
```

Префиксный/постфиксный инкремент/декремент

- Напоминание:

```
int a = 10;  
  
// Префиксный: сначала прибавить, потом вернуть значение  
int b = ++a;  
// Здесь a = 11, b = 11  
  
// Постфиксный: сначала вернуть значение, потом прибавить  
int c = a++;  
// Здесь a = 12, c = 11
```

- Как их перегрузить по-отдельности в C++?

Префиксный/постфиксный инкремент/декремент

```
class BigInt {  
    ...  
public:  
    // Префиксный  
    BigInt& operator++();  
  
    // Постфиксный (возвращает копию!)  
    // int - фиктивный параметр, не используется  
    BigInt operator++(int) {  
        BigInt copy(*this);  
        // Или this->operator++();  
        ++(*this);  
        return copy;  
    }  
};
```

Операторы сравнения

- В общем случае, нужно реализовать 6 штук, 4 из которых в одну строку:

```
bool operator==(const BigInt& a, const BigInt& b) { ... }
bool operator<(const BigInt& a, const BigInt& b) { ... }

bool operator!=(const BigInt& a, const BigInt& b) {
    return !(a == b);
}

bool operator>(const BigInt& a, const BigInt& b) {
    return b < a;
}

bool operator<=(const BigInt& a, const BigInt& b) {
    return !(b < a);
}

bool operator>=(const BigInt& a, const BigInt& b) {
    return !(a < b);
}
```

Операторы сравнения

- Можно даже `a == b` выразить через `a < b`:

```
bool operator==(const BigInt& a, const BigInt& b) {  
    return !(a < b) && !(b < a);  
}
```

Операторы сравнения

- Можно даже `a == b` выразить через `a < b`:

```
bool operator==(const BigInt& a, const BigInt& b) {  
    return !(a < b) && !(b < a);  
}
```

- Такой семантики придерживаются некоторые стандартные алгоритмы и структуры данных

Операторы сравнения

- Можно даже `a == b` выразить через `a < b`:

```
bool operator==(const BigInt& a, const BigInt& b) {  
    return !(a < b) && !(b < a);  
}
```

- Такой семантики придерживаются некоторые стандартные алгоритмы и структуры данных
- Часто такая реализация менее эффективна, чем сравнение напрямую

- Начиная с C++20 есть т.н. *three-way comparison* (он же – *spaceship operator*): `a <=> b`

- Начиная с C++20 есть т.н. *three-way comparison* (он же – *spaceship operator*): `a <=> b`
- Возвращает особый тип из трёх значений (меньше, равно, больше), например `std::strong_ordering`

Default и three-way сравнения

- Начиная с C++20 есть т.н. *three-way comparison* (он же – *spaceship operator*): `a <=> b`
- Возвращает особый тип из трёх значений (меньше, равно, больше), например `std::strong_ordering`
- Можно определить только `a == b` и `a <=> b`, остальные компилятор выведет за нас

Оператор приведения типа

- Что, если я хочу, чтобы мой класс можно было приводить к другому типу?

```
B b;  
A a = b;  
A a(b);  
funcOfA(b);
```

Оператор приведения типа

- Что, если я хочу, чтобы мой класс можно было приводить к другому типу?

```
B b;  
A a = b;  
A a(b);  
funcOfA(b);
```

- Два варианта: сделать у **A** конструктор от **B**, или сделать у **B** оператор приведения типа к **A**

Оператор приведения типа

- Что, если я хочу, чтобы мой класс можно было приводить к другому типу?

```
B b;  
A a = b;  
A a(b);  
funcOfA(b);
```

- Два варианта: сделать у **A** конструктор от **B**, или сделать у **B** оператор приведения типа к **A**
- **NB:** Если **A** – примитивный тип, ему не получится добавить конструктор, и остаётся только второй вариант

Оператор приведения типа

```
class BigInt {  
public:  
    // Оператор приведения типа BigInt -> double  
    operator double() const {  
        ...  
    }  
};  
  
BigInt x = 10;  
x = pow(x, 10000);  
  
double y = x;
```

Оператор приведения типа

- Можно пометить как **explicit** и разрешить только явное приведение типов:

```
class BigInt {  
public:  
    explicit operator double() const;  
};  
  
BigInt x = 10;  
  
// Ошибка: неявное приведение  
double y = x;  
  
// ОК: явное приведение  
double z = (double)x;
```


Оператор приведения типа

- Можно пометить как **explicit** и разрешить только явное приведение типов:

```
class BigInt {  
public:  
    explicit operator double() const;  
};  
  
BigInt x = 10;  
  
// Ошибка: неявное приведение  
double y = x;  
  
// ОК: явное приведение  
double z = (double)x;
```

- Лучше всегда пометать **explicit**, цепочки неявных преобразований типов – большая головная боль

- До C++11 приходилось писать странные трюки, чтобы класс можно было использовать аналогично `bool` – использовать в `if`, `while`, `&&` и так далее (см. *safe bool idiom*)

- До C++11 приходилось писать странные трюки, чтобы класс можно было использовать аналогично `bool` – использовать в `if`, `while`, `&&` и так далее (см. *safe bool idiom*)
- В C++11 появились `explicit` операторы приведения типов, и теперь всё это делается просто как `explicit operator bool()`

Operator bool

```
class BigInt {
public:
    explicit operator bool() const {
        return (*this) != 0;
    }
};

BigInt x = 10;

// Ошибка: неявное приведение типов
bool b = x;

// А тут можно :)
if (x) {
    ...
}
```